

The Workflow Data Manifest

FRIDAY AI Education — Episode 024 · 2026-07-10 · Printable Companion

Knowing What Your Workflow Knows — Before It Acts On It

FRIDAY AI Education — Episode 024 · Printable Companion Paper

Abstract

Every automated workflow is a bet on its inputs. When an AI-assisted process drafts an email, summarizes a deal, or answers a question about a company, the quality and legitimacy of the output are determined less by the model and more by the data it was allowed to touch: where that data came from, who owns it, how old it is, and whether it was permitted to be there at all. Most workflow failures that matter — the embarrassing ones, the legal ones, the trust-destroying ones — are data provenance failures, not reasoning failures. This paper introduces the **data manifest**: a short, explicit declaration, written before a workflow runs, of what data the workflow uses, who owns each source, how fresh it must be, where the privacy boundary sits, and what each source may be used for. The manifest is a one-page discipline, not a compliance program. We build it up from first principles, walk through three concrete cases — a public-content knowledge base built from Galoy's published material, a CRM-driven workflow, and meeting notes — and connect the practice to the *Govern* function of the NIST AI Risk Management Framework. The goal is that by the end, you can write a defensible manifest for any bounded workflow in under thirty minutes, and recognize instantly when a workflow is running without one.

1. The Problem: Workflows Fail Upstream

Consider a plausible near-future scene. An AI workflow you built drafts a follow-up email to a prospective customer. The draft is fluent, well-structured, and correctly personalized. It also references a pricing concession that was discussed in an internal strategy meeting, mentions a fact about the prospect's company that came from a paywalled database whose license prohibits redistribution, and cites a statistic from a blog post that was retracted eight months ago.

The model did nothing wrong. It reasoned competently over the context it was given. The failure happened before the model ever ran: somebody — probably nobody, which is the real problem — decided that internal meeting notes, licensed data, and stale web content all belonged in the same context window, with no labels distinguishing them.

This is the central insight of this episode, and it is worth stating plainly:

Most consequential AI workflow failures are input failures. The model amplifies whatever data discipline you have — including none.

When people evaluate AI systems, they instinctively focus on the model: is it smart enough, does it hallucinate, which vendor is best. These are real questions, but they are the questions vendors compete on, which means they improve without your involvement. The questions that do not improve on their own are the ones about *your* data: what goes in, under what terms, and with what guarantees. That is where an operator's leverage lives, and that is what the data manifest governs.

There is a second, more strategic reason to care. If you intend to build companies where AI workflows do real operational work — touching customers, money, or reputation — you will eventually sit across a table from someone asking, "How do you know your system won't leak our data or act on stale information?" The people who can answer that question specifically get the contract, the partnership, or the investment. The people who answer with "we use a very good model" do not. A data manifest is, among other things, the artifact that lets you answer that question in one page.

2. The Core Concept: What a Data Manifest Is

A **data manifest** is a short, explicit declaration attached to a workflow that answers five questions about every data source the workflow touches:

1. **What is it?** — The source, named precisely enough that someone else could find it.
2. **Who owns it?** — The party with rights over the data, which is often not you.
3. **How fresh is it?** — When it was last updated, and how stale it can be before it becomes wrong.
4. **Where is the privacy boundary?** — What audience the data was originally scoped to, and whether this workflow keeps it inside that scope.
5. **What is it allowed to be used for?** — The permitted purposes, which are narrower than "anything the model can do with it."

The word *manifest* is borrowed deliberately from shipping. A ship's manifest lists every item of cargo: what it is, who it belongs to, where it boarded, where it may be unloaded. Nobody considers this bureaucratic overreach; a ship carrying unlisted cargo is called a smuggling vessel. A workflow consuming unlisted data is the same thing with better branding.

Three properties make a manifest useful rather than ceremonial:

It is written before the workflow runs. A manifest reconstructed after an incident is called an apology. The value comes from the act of declaring, because declaring forces you to notice the sources you cannot honestly describe.

It is per-workflow, not per-company. A company-wide data policy is too abstract to catch anything. "We respect customer privacy" prohibits nothing in particular. "This follow-up-email workflow reads CRM contact records and public web pages, and nothing else" prohibits something extremely particular, and can be checked.

It is short. A manifest for a bounded workflow is a table with five to ten rows. If your manifest runs to twenty pages, either your workflow is too big to be one workflow, or you have written a legal document instead of an engineering artifact. Both are fixable; neither is a manifest.

Note what a manifest is *not*: it is not a data catalog (an inventory of everything you have), not a privacy policy (a promise to outsiders), and not an access-control system (an enforcement mechanism). It sits upstream of all three. The manifest is the *decision*; catalogs, policies, and permissions are ways of recording and enforcing decisions. You cannot enforce a decision that was never made.

3. The Mechanism: Five Questions, Taken Seriously

Each of the five manifest questions looks trivial until you try to answer it honestly for a real workflow. This section takes them one at a time, because the difficulty — and the value — is in the details.

3.1 Source: Name the Data Precisely

The first discipline is specificity. "Customer data" is not a source. "The contacts and deals tables in the CRM, read-only, excluding the free-text notes field" is a source. The test is whether another person, reading your manifest, could locate exactly what the workflow reads and — equally important — determine what it does *not* read.

Vagueness here is not a style problem; it is where scope creep enters. A workflow whose manifest says "company documents" will, six months later, be reading the board minutes, because board minutes are company documents and nobody said otherwise. Precision at declaration time is what makes "no" possible later.

Specificity also forces you to confront the *transitive* sources — the data hiding inside your data. Meeting notes are one source on paper, but they contain fragments of many others: things clients said, numbers from financial reports, opinions of colleagues who did not know they were being transcribed. A manifest that lists "meeting notes" without acknowledging this is precise about the container and silent about the cargo. We return to this in Section 4.3, because meeting notes are the single most underestimated source in operational AI.

3.2 Ownership: Who Holds the Rights

Ownership is the question operators most often skip, because possession feels like ownership. It is not. Data in your systems routinely belongs, in the senses that matter, to someone else:

- **Your own data.** Content you created: your published essays, your product documentation, your internal analyses. You have the broadest rights here, though even self-authored data can carry obligations (an internal analysis of a partner's business is yours to hold, not necessarily yours to publish).
- **Customer data.** Information customers gave you for a purpose: contact details, usage records, support conversations. You are a *custodian*. The data sits in your database, but the meaningful rights — and the reasonable expectations — remain with the customer. Using a support ticket to resolve the support issue is custodianship; using it to train a public-facing model is a unilateral renegotiation of the deal.
- **Third-party public data.** Published material owned by others: blog posts, talks, documentation, filings. Public means *readable*, not *free for all uses*. You may read a company's blog; whether you may republish it wholesale, or build a commercial product on verbatim copies of it, is a different question with a different answer.
- **Licensed data.** Data you pay for — market data, enrichment APIs, research databases — comes with a contract that specifies permitted uses, and those contracts are typically written by people who anticipated you would want to do more than you paid for.
- **Employee and counterparty data.** What colleagues say in meetings, what partners share under NDA, what candidates submit in applications. Each carries an implicit or explicit scope set by the context in which it was shared.

The manifest question is not "can I access this?" — you obviously can, or the question would not arise. The question is "whose data is this, and does my use match what they agreed to or could reasonably expect?" When a workflow blends five sources into one output, the output inherits the *most restrictive* obligations of any input. This is the one rule of data blending worth memorizing: restrictions compose upward. Mix one confidential source into ten public ones and the blend is confidential.

3.3 Freshness: When Data Quietly Becomes Wrong

Freshness is the least dramatic of the five questions and the one most likely to bite you in ordinary operation, because stale data does not announce itself. A stale fact looks exactly like a fresh fact. It sits in the retrieval index with the same confidence it had the day it was true.

The mechanism to internalize: **data does not decay visibly, but the world it describes moves.** Pricing changes, people change roles, products get renamed, positions get reversed. A model retrieving from a six-month-old snapshot will state six-month-old facts in the fluent present tense, and the fluency is precisely what makes staleness dangerous — a confident wrong answer outperforms a hesitant one at getting believed.

The manifest handles this with two declarations per source:

- **Last verified:** when the data was last confirmed current, or when the snapshot was taken.
- **Maximum acceptable age:** how stale this source can be before the workflow should stop trusting it, flag it, or refresh it.

The second number varies enormously by content type, and setting it is a judgment call you should make consciously rather than by default. A company's founding story is effectively immortal. Its team page decays in months. Its pricing can decay in a week. A useful heuristic: ask *"what is the cost of the workflow asserting this after it changed?"* and set the maximum age where that cost becomes unacceptable. For a historical anecdote the cost is nil; for a price quoted to a customer the cost is a concrete broken promise.

Freshness also changes how you build. Once a source has a declared maximum age, someone or something must be responsible for refreshing it — a re-crawl schedule, a sync job, a human review. An unrefreshed source with a freshness requirement is a liability with a due date. This is unglamorous plumbing, and it is where most knowledge-base projects quietly fail: not at launch, when everything is fresh by construction, but eighteen months later, when the index has become a museum that answers questions as if it were still opening day.

3.4 Privacy Boundary: The Original Audience Travels With the Data

Every piece of data was created with an intended audience, and that intended audience is the data's **privacy boundary**. A public blog post was scoped to the world. A CRM record was scoped to your company's commercial team. A candid remark in a leadership meeting was scoped to the people in the room. The boundary is not written on the data — which is exactly why it must be written in the manifest.

The mechanism that makes this an AI problem rather than a general filing problem is that AI workflows are boundary-crossing machines by design. Their entire value proposition is moving information from where it sits to where it is useful: retrieve, summarize, draft, answer. Every retrieval is a potential boundary crossing. A human assistant has social instincts about what not to repeat outside the room; a retrieval pipeline has cosine similarity. If the candid remark is semantically relevant to the query, it will be retrieved, and the model will faithfully weave it into a customer-facing draft, because nothing told it not to. The model is not indiscreet. It is *undiscriminating*, which is worse, because indiscretion at least implies awareness of a line.

The manifest discipline is to declare, per source, two audiences:

- **Original boundary:** who this data was scoped to when created.
- **Workflow output boundary:** who can see what this workflow produces.

The rule is then mechanical: **data can flow to an equal or narrower audience without ceremony; any flow to a broader audience is an event that requires an explicit decision.** Internal meeting notes feeding an internal briefing document — fine, the boundary is preserved. The same notes feeding an email to a customer — that is a boundary crossing, and it must be either deliberately approved or structurally prevented. In practice, structural

prevention beats vigilance: the most reliable way to keep meeting notes out of customer emails is for the customer-email workflow's manifest to simply not include meeting notes as a source. What is not retrieved cannot leak. Separation at the manifest level is cheaper and more reliable than filtering at the output level, in the same way that not carrying a secret is easier than remembering not to say it.

3.5 Allowed Use: Purpose Is Not Transitive

The final question is the subtlest: even data you legitimately hold, within its boundary, is typically permitted for some purposes and not others. The formal term for the underlying principle is **purpose limitation** — data collected for one purpose should not be silently repurposed for another — and it appears in essentially every serious data-protection regime, because it encodes a real intuition about consent. A customer who gave you their email to receive invoices did not thereby agree to receive your newsletter, fuel your lookalike-audience model, or star as a named example in your sales deck.

The manifest declaration is one line per source: *this data may be used for X and Y; notable exclusions are Z*. The exclusions matter more than the inclusions, because exclusions are what an automated workflow will violate by default. A model given CRM access and asked to "write something compelling for the prospect" will happily mine another customer's private usage statistics for a persuasive comparison. It is not violating its instructions. It was never given any.

Allowed use is also where *aggregation* deserves a sentence of respect. Individually permissible uses can compose into an impermissible one: any single meeting note is innocuous; a year of meeting notes, queried systematically, is a surveillance record of what every colleague said and when. Workflows make aggregation effortless, so the manifest for any workflow that reads across many records over time should say explicitly what aggregate questions it will and will not serve.

4. Three Worked Examples

Theory earns its keep in cases. Here are three, in ascending order of difficulty.

4.1 Galoy's Public Footprint: The Easy Case (Which Still Has Edges)

Galoy is an open-source Bitcoin banking-infrastructure company with a substantial public footprint: an open-source codebase, technical documentation, blog posts, conference talks, podcast appearances. Suppose you build a knowledge base over this public body of work — the pattern of making a company's published corpus searchable, citable, and operationally useful.

The manifest almost writes itself, which is exactly why it is the right first case:

- **Source:** Enumerated public URLs — the docs site, the blog, the GitHub repositories, specific talk recordings. Enumerated, not "whatever the crawler finds": a crawler pointed at "Galoy content" will eventually ingest a third-party forum thread *about* Galoy and present speculation as documentation.
- **Ownership:** Galoy and the individual speakers own the content. The open-source code carries an explicit license — the one place in this case where permitted use is written down formally. The blog posts and talks are public to *read*; your right to excerpt, summarize, and cite them is solid, your right to republish them wholesale is not.
- **Freshness:** The sharp edge of this case. Bitcoin infrastructure evolves quickly; documentation describes the software as of a date. The manifest should record crawl dates per source and set an explicit maximum age, with a re-crawl schedule to match. A knowledge base that answers from a two-year-old snapshot of a fast-moving

project is not a knowledge base; it is a well-indexed misunderstanding.

- **Privacy boundary:** Genuinely clean — everything was published to the world, so no downstream use can broaden the audience. This is the structural reason public-content knowledge bases are the correct *first* wedge for anyone building in this space: you get to develop the retrieval, citation, and freshness machinery on data that cannot leak, before pointing the same machinery at data that can.
- **Allowed use:** Search, summarization with citation, question-answering that attributes claims to sources. Notable exclusion: presenting synthesized inferences as Galoy's stated positions. "Galoy's docs say X (link)" is a citation; "Galoy believes Y," where Y is your model's interpolation across three blog posts, is ventriloquism.

The lesson of the easy case: even the friendliest possible data — public, permissively licensed, boundary-free — still needs three of the five manifest questions answered carefully. There is no data so safe that the manifest is empty.

4.2 CRM Data: The Custodianship Case

Now the middle difficulty: a workflow that reads your CRM to draft follow-up emails, prepare call briefings, or summarize pipeline. Every company has this data; nearly every AI-workflow vendor is racing to plug into it.

- **Source:** Specific objects and fields — contacts, deals, activity logs. The free-text notes field deserves its own line in the manifest, because it is a different animal wearing the same schema: structured fields contain what the system was designed to hold, while notes fields contain whatever a salesperson felt like typing, including health details, personal circumstances, and unvarnished opinions of the people involved. If you include notes as a source, you have included all of that.
- **Ownership:** You are the custodian, not the owner. The contact details were provided by prospects and customers for the purpose of doing business with you. Layered on top: your salespeople authored the notes, your company owns the database, and data-protection law gives the individuals in it ongoing rights regardless of whose server it sits on.
- **Freshness:** CRM data decays person by person. Contacts change jobs; champions leave; titles rot. A briefing workflow should treat contact records older than some threshold as "verify before relying" rather than ground truth — a small manifest note that prevents the specific embarrassment of an email opening warmly to someone who left the company last spring.
- **Privacy boundary:** Original scope is your commercial team. An internal briefing preserves it. Any workflow that moves CRM-derived content *outward* — into emails, proposals, marketing — crosses the boundary and needs the crossing designed: which fields may appear in external text, and which may never. The clean pattern is two manifests: an internal-facing workflow with broad field access, and an external-facing one with a deliberately impoverished source list. Separation over vigilance, again.
- **Allowed use:** Serving each customer relationship with that customer's own data. The canonical violation is cross-record leakage: using Customer A's confidential usage data to impress Prospect B. Every record was given to you in a separate act of trust; the CRM's flat table structure erases those separations, and the manifest must reinstate them.

The lesson: CRM data is where "we have this data" and "we may use this data for that" diverge most visibly, and where a workflow without a manifest will do something indefensible within its first hundred runs — competently, fluently, and without anyone deciding it should.

4.3 Meeting Notes: The Hard Case

Meeting notes — and their modern successor, automatic meeting transcripts — are the hardest common source, because they fail all five questions at once in ways that are easy to miss.

- **Source:** A meeting note is a *container of other people's data*. Inside it: client statements, colleague opinions, numbers quoted from confidential documents, half-formed positions that the speaker revised the next day. Listing "meeting notes" as one source understates the ingestion by an order of magnitude.
- **Ownership:** Radically mixed and mostly implicit. The client who said something candid in a call did not assign you rights to it; the colleague who floated a bad idea did not consent to its preservation. Nobody in the room signed anything about the note's downstream life.
- **Freshness:** Uniquely treacherous, because meeting notes capture *positions*, and positions are the fastest-decaying data type there is. "We've decided to raise prices in Q3" may have been reversed in the following week's meeting — possibly in a meeting that wasn't transcribed. A retrieval system will surface the recorded decision, not the unrecorded reversal, and will do so indefinitely. Notes are true only as *records that something was said on a date*; a manifest should force workflows to treat them as dated utterances, never as current facts.
- **Privacy boundary:** The original audience was *the people in the room, that day*. Every retrieval by anyone else is a boundary crossing. Transcription tools quietly transformed a social norm — what's said in the meeting stays roughly in the meeting — into a searchable archive, without anyone renegotiating the norm. When people realize their offhand remarks are retrievable by workflows they've never heard of, they stop saying anything worth transcribing. The chilling effect is the real cost, and it arrives silently.
- **Allowed use:** Deserves the narrowest grant in your whole manifest portfolio: continuity for the participants (what did we discuss, what did I commit to), scoped to the people who were in the room. Explicit exclusions: feeding external communications, and aggregate queries across people and time ("summarize everything Sarah has said about the budget this year" is a dossier, not a summary).

The lesson of the hard case: the sources that feel most natural to ingest — they're right there, they're yours, they're so useful — are often the ones where every manifest question returns the most restrictive answer. Difficulty is uncorrelated with convenience.

4.4 The Three Cases Side by Side

Manifest question	Galoy public sources	CRM data	Meeting notes
Source	Enumerated public URLs, per-source crawl dates	Named objects/fields; notes field listed separately	Containers of many parties' statements
Ownership	Galoy / speakers; license governs code	You as custodian; customers retain rights	Mixed, implicit, never negotiated
Freshness	Re-crawl on schedule; fast-moving domain	Decays per contact; verify stale records	Positions decay in days; treat as dated utterances
Privacy boundary	None — published to the world	Internal commercial team; outward flow needs design	The room, that day; every reuse is a crossing
Allowed use	Cited search & summary; no ventriloquism	Per-relationship service; no cross-record leakage	Participant continuity only; no dossiers
Overall difficulty	Easy — the right first wedge	Medium — custodianship discipline	Hard — most restrictive on every axis

5. Where This Sits in the Bigger Picture: Governance, and the NIST Frame

The data manifest is not an invention of this paper; it is the workflow-scale expression of an idea that mature risk frameworks state at organizational scale. The **NIST AI Risk Management Framework** — the reading anchor for this episode — organizes AI risk management into four functions: *Govern, Map, Measure, Manage*. The one that matters

here is **Govern**, and its relevant demand is simple to state: an organization using AI should have policies, accountability, and documented practices for the data its systems consume — provenance, ownership, and permitted use should be *decided and recorded*, not emergent.

Two things are worth taking from the NIST framing beyond the checklist.

First, **Govern is described as a cross-cutting function** — it is not a phase you complete before the real work, but a property maintained throughout the system's life. This matches the manifest's most important operational feature: it is a living document. When a workflow gains a new source, the manifest changes *first*; a manifest that no longer matches the pipeline is worse than none, because it provides confidence without coverage.

Second, the framework's vocabulary is the vocabulary of serious rooms. Enterprises, regulators, and sophisticated partners increasingly assess AI systems in these terms. An operator who can say "here is the data manifest for this workflow — sources, ownership, freshness windows, boundaries, permitted uses" is speaking the governance language fluently at exactly the scale a counterparty can verify. This is the honest sense in which a one-page discipline is also a competitive asset: trust is the product, and the manifest is inspectable evidence of it.

A note on proportion, because governance talk attracts maximalists: the manifest is deliberately the *minimum viable* version of data governance. It is not a data catalog, a DPIA, or an enterprise metadata platform. For an operator running bounded workflows, five questions per source, answered honestly on one page, captures most of the value of those heavier instruments at roughly one percent of the cost. Scale the machinery when the workflows scale; start with the page.

Finally, the manifest slots into a larger anatomy worth keeping in view. A trustworthy operational AI system needs context, memory, tools, permissions, evaluations, audit, and human approval gates. The manifest is the *data* leg of that structure — the permissions-and-audit logic applied to inputs. A system with excellent action-permissions but unmanifested data is a polite agent with unvetted knowledge: it will ask before sending the email, but nobody vetted what it was allowed to know while writing it.

6. Failure Modes: How Manifests Go Wrong

A tool is only understood when its failure modes are. Four dominate.

The manifest that drifts. The workflow gains a source — someone plugs in a new folder, a new integration, a new index — and the manifest is not updated. This is the most common failure and the most corrosive, because a stale manifest actively misinforms: it tells auditors, partners, and future-you that the workflow is narrower than it is. Mitigation is procedural and cheap: adding a source *is* editing the manifest; there is no separate step to forget.

The manifest as theater. Written once to satisfy a checklist, never consulted, disconnected from the pipeline it describes. The tell is a manifest with no exclusions — a document that only lists what is used, and never what was considered and rejected, was probably written after the fact. Real manifests have scar tissue: sources someone wanted and was denied.

The over-engineered manifest. The opposite failure: forty fields per source, a review committee, a quarterly attestation cycle — for a workflow that drafts internal briefings. The cost of this failure is not the wasted effort; it is that the next workflow skips the manifest entirely because the last one was so painful. Keep the instrument proportional to the risk, or lose the practice.

The manifest without enforcement. The document says "meeting notes are excluded," but the retrieval index still contains them, so the workflow reads them anyway. A manifest is a decision; decisions need *some* mechanical

expression — a source allowlist in the pipeline, separate indexes per boundary, a check in the retrieval layer. The declaration and the enforcement can be simple, but they must be connected. Paper does not filter queries.

The common thread: the manifest fails when it stops being *load-bearing* — when the pipeline would behave identically whether the document existed or not. The test of a living manifest is that deleting a line from it would change what the workflow can do.

7. Vocabulary

Terms worth owning precisely — these are the words that signal fluency in serious conversations.

Data manifest. A per-workflow declaration of every data source used, covering source identity, ownership, freshness, privacy boundary, and allowed use. Written before the workflow runs; updated when the workflow changes.

Source ownership. The identification of who holds rights over a data source — distinct from who possesses or stores it. Key distinction: *owner* versus *custodian*. You own what you created; you are custodian of what others entrusted to you.

Custodianship. Holding data on behalf of its owner, with obligations to use it only within the owner's reasonable expectations. Describes most customer data a company holds.

Freshness. The currency of data relative to the world it describes, typically expressed as *last verified* plus *maximum acceptable age*. Stale data is dangerous precisely because it is indistinguishable, in form, from fresh data.

Privacy boundary. The audience a piece of data was originally scoped to. Data may flow to equal or narrower audiences freely; flows to broader audiences are events requiring explicit decision.

Boundary crossing. Any movement of data beyond its original audience. AI retrieval systems perform these mechanically unless structurally prevented.

Allowed use. The set of purposes for which a source may legitimately be used — always narrower than the set of purposes it is *useful* for. Best specified with explicit exclusions.

Purpose limitation. The principle that data collected for one purpose should not be silently repurposed for another; the legal-and-ethical foundation under "allowed use."

Provenance. The recorded origin and chain of custody of a piece of data — where it came from, when, and through what transformations. Manifests are provenance discipline applied at the workflow level.

Aggregation risk. The phenomenon where individually harmless data items compose into a harmful whole (one meeting note vs. a queryable year of them). Workflows create this risk by making aggregation effortless.

Retrieval augmented generation (RAG). The architecture where a model's answers are grounded in documents retrieved at query time. Relevant here because the retrieval index *is* the enforcement point for most manifest decisions: what is not indexed cannot be retrieved, and what cannot be retrieved cannot leak.

8. Reading Guide

NIST AI Risk Management Framework — Core and Profiles (Govern function)

<https://airc.nist.gov/AI_RMF_Knowledge_Base/AI_RMF/Core_And_Profiles>

How to read it without drowning — this is a framework document, so it reads like one; budget forty-five minutes and go in with a filter:

1. **Skim the structure first.** Understand the four functions — Govern, Map, Measure, Manage — and how they relate. Ten minutes. The architecture is the insight; the framework's claim is that trustworthy AI is an organizational practice, not a model property.
2. **Read the Govern function closely.** This is the episode's anchor. As you read each Govern category, ask one question: *what is the workflow-scale version of this organizational demand?* You will find that a surprising number of them reduce, at the scale of one workflow, to a line in a data manifest. That mapping — big framework to small artifact — is the skill being built.
3. **Skim Map; skip Measure and Manage for now.** Map (establishing context, identifying what data and systems are in play) is the manifest's nearest cousin and worth five minutes. Measure and Manage belong to future episodes on evaluations and operations.
4. **Do not read it as a compliance checklist.** Read it as a preview of the questions sophisticated counterparties will ask. The useful exercise is not "are we compliant" — the RMF is voluntary — but "could I answer this question, specifically, for one workflow I run?" Where the answer is no, you have found next week's thirty-minute task.

9. Walking Questions

Take these on the next walk. They are ordered from comprehension to application.

1. **Explain it forward.** Could you explain to a smart non-technical friend why "the model did nothing wrong" and "the workflow failed badly" can both be true of the same incident? If the explanation doesn't naturally route through the inputs, reread Section 1.
2. **Find the blur.** Of the five manifest questions — source, ownership, freshness, boundary, allowed use — which one could you apply confidently right now, and which is still blurry? (For most people it is allowed use; the distinction between "have," "may hold," and "may use for this" takes practice.)
3. **Locate it at home.** Where in your own systems is there data flowing into AI workflows today with no manifest? Be honest about meeting transcripts specifically — Section 4.3 was written the way it was for a reason.
4. **The most restrictive input.** Pick any workflow you run or plan to run. Which single source in it carries the most restrictive obligations — and does the workflow's output currently respect them, by design or by luck?
5. **A bounded test.** What is one small workflow — a public-content knowledge base is the natural candidate — where you could write a complete five-question manifest in thirty minutes this week, and have the pipeline actually enforce it? Not a program. One page, one workflow, actually load-bearing.

10. One-Sentence Takeaway

Before a workflow is allowed to act, someone must have written down what it is allowed to know — where each source came from, who owns it, how fresh it is, who was meant to see it, and what it may be used for — because a model will fluently amplify whatever data discipline you have, including none.